



**ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ
ГОРОДА МОСКВЫ**
**Государственное бюджетное профессиональное
образовательное учреждение города Москвы**
«Колледж малого бизнеса № 4»
(ГБПОУ КМБ № 4)

GIT

Специальность: 09.02.07 Информационные системы и программирование

Форма обучения: очная

Студент(ка): Перепёлкина Анфиса Александровна

Группа: ИПО 21.24

Руководитель: Александр Сергеевич Рыбаков

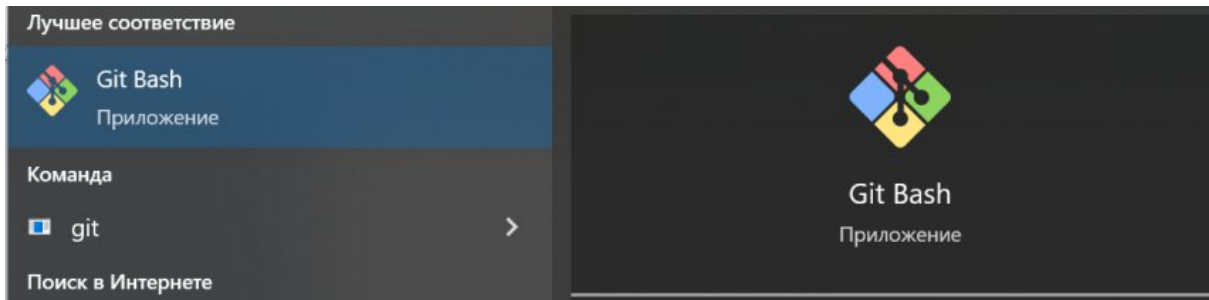
Отчётная работа защищена с оценкой «___» _____

Москва, 2025 г.

Оглавление

Лабораторная работа №2	3
Контрольные вопросы:	4
Лабораторная работа №3	5
Самостоятельная работа:	10
Контрольные вопросы:	14
Лабораторная работа №4	15
Самостоятельная работа:	17
Контрольные вопросы:	20
Лабораторная работа №5	21
Контрольные вопросы:	22
Лабораторная работа №6	23
Контрольные вопросы:	23
Лабораторная работа №7	24
Самостоятельная работа:	33
Контрольные вопросы:	43

Лабораторная работа №2



Линукс команды

Ls для вывода списка всех файлов

```
User@S_BMESH_20 MINGW64 ~  
$ ls  
'3D Objects'          IntelGraphicsProfiles  Pictures              ntuser.ini  
AppData               Link$                  PrintHood            source  
'Application Data'   'Local Settings'      Recent               'Мои документы'  
'Cisco Packet Tracer 8.2.0' Music                  'Saved Games'        Учебная.mdf  
Contacts              NTUSER.DAT             Searches             Учебная_log.ldf  
Cookies               NTUSER.DAT{53b39e88-18c4-11ea-a811-000d3aa4692b}.TM.btf  SendTo               шаблоны  
Desktop               NTUSER.DAT{53b39e88-18c4-11ea-a811-000d3aa4692b}.TM.container000000000000000001.regtrans-ms  Videos              'главное меню'  
Documents             NTUSER.DAT{53b39e88-18c4-11ea-a811-000d3aa4692b}.TM.container000000000000000002.regtrans-ms  'VirtualBox VMs'    ntuser.dat.LOG1  
Downloads             NetHood                VirtualBox VMs       ntuser.dat.LOG2  
Favorites             OneDrive
```

Cd для вывода списка всех файлов

```
User@S_BMESH_20 MINGW64 ~  
$ cd/d/tmp  
bash: cd/d/tmp: No such file or directory  
  
User@S_BMESH_20 MINGW64 ~  
$ cd c:/d/TMP  
  
User@S_BMESH_20 MINGW64 /c/d/TMP  
$ |
```

PWD – вывод директории

Регистрация в сети гит

```
User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp/sm_3laba (master)  
$ git config --global user.name "fisa"  
  
User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp/sm_3laba (master)  
$ git config --global user.email "anfisasunny@gmail.com"
```

Инициализация репозитория и проверка пустой он или нет(через –a вывелись скрытые файлы)

```

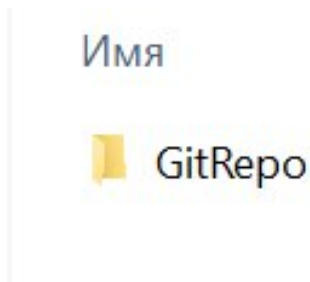
User@S_BMESH_20 MINGW64 /c/d/TMP
$ git init
Initialized empty Git repository in c:/d/TMP/.git/

User@S_BMESH_20 MINGW64 /c/d/TMP (master)
$ ls -a
./ ../ .git/

User@S_BMESH_20 MINGW64 /c/d/TMP (master)
$ |

```

Создаем папку GitRepo в диске d и переносим туда папку tmp



Переходим к папке tmp по новому пути и смотрим текущий статус

```

User@S_BMESH_20 MINGW64 /c/d/GitRepo/TMP (master)
$ ls -a
./ ../ .git/

User@S_BMESH_20 MINGW64 /c/d/GitRepo/TMP (master)
$ git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)

User@S_BMESH_20 MINGW64 /c/d/GitRepo/TMP (master)

```

GitBash – консольный интерфейс(линуксный) взаимодействия с ядром

Повседневные команды :

Git init – инициализация проекта

Git status – просмотр текущего статуса репозитория

Global – для настройки общих параметров

Создаем файл README.md в папке репозитория

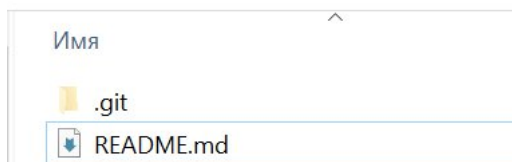
Контрольные вопросы:

1. **Что такое GitBash?** GitBash — эмулятор командной строки (CLI) для работы с Git в среде Windows. Объединяет команды Unix и инструменты Git.

2. **Для чего нужна команда ls?** Команда LS выводит список файлов и папок в текущей директории. Аналог команды DIR в Windows.
3. **Как сменить директорию?** Для смены директории используется команда CD <ПУТЬ>, например: CD .
4. **Как отобразить текущую директорию?** Чтобы отобразить текущую директорию, нужно ввести команду PWD.
5. **Для чего задаются настройки Git?** Настройки Git задают:
 - имя и email пользователя;
 - поведение по умолчанию для работы с репозиториями;
 - параметры форматирования и другие опции (например, GIT CONFIG --GLOBAL USER.EMAIL "YOUREMAIL")
6. **Как применить одинаковые настройки для всех репозиториях?** Чтобы применить настройки для всех репозиториях, нужно использовать флаг --GLOBAL в команде GIT CONFIG.

Лабораторная работа №3

22H2 (C:) > d > GitRepo > TMP



Git add – передает файл в систему git (файл теперь отслеживается системой), в подготовленное состояние (staged)

После добавления README проверяем заново статус репозитория и видим, что появился неотслеживаемый файл

```
User@S_BMESH_20 MINGW64 /c/d/GitRepo/TMP (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
      README.txt

nothing added to commit but untracked files present (use "git add" to track)
User@S_BMESH_20 MINGW64 /c/d/GitRepo/TMP (master)
$ |
```

Командой `Git add .` и “показываем” гиту наш новый файл

```
User@S_BMESH_20 MINGW64 /c/d/GitRepo/TMP (master)
$ git add .

User@S_BMESH_20 MINGW64 /c/d/GitRepo/TMP (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md
```

Коммитим наши действия, с обязательным комментарием к нему

`Git commit` - - для создания коммита (-m далее пишем текст нашего коммита в “”)

`Git commit` – попадаю в текстовый редактор `vim`

`Esc -> :q`

`->: wq` – Сохранить файл и выйти

`->:x` – Выйти без сохранения

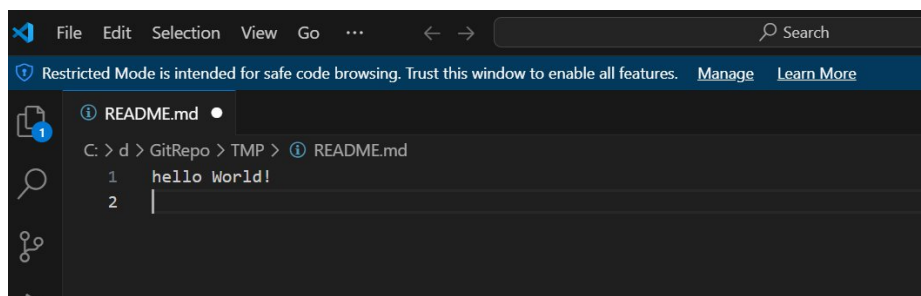
```
User@S_BMESH_20 MINGW64 /c/d/GitRepo/TMP (master)
$ git commit -m "Добавили новый файл README.md"
[master (root-commit) 6da5f2d] Добавили новый файл README.md
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 README.md

User@S_BMESH_20 MINGW64 /c/d/GitRepo/TMP (master)
$ |
```

Заново
проверяем
статус
репозитор
ия

```
User@S_BMESH_20 MINGW64 /c/d/GitRepo/TMP (master)
$ git status
On branch master
nothing to commit, working tree clean
```

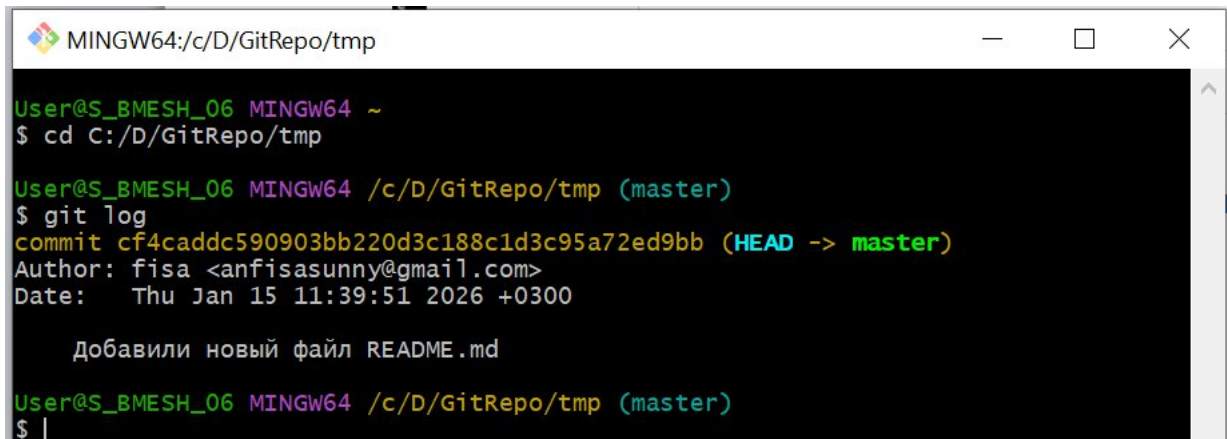
Пишем в файле `README.md` “hello World!”



The screenshot shows a code editor window with the file `README.md` open. The file path is `C:\> d > GitRepo > TMP > README.md`. The content of the file is:

```
1 hello World!
2 |
```

Git log - просмотр всех коммитов



```
MINGW64:/c/D/GitRepo/tmp

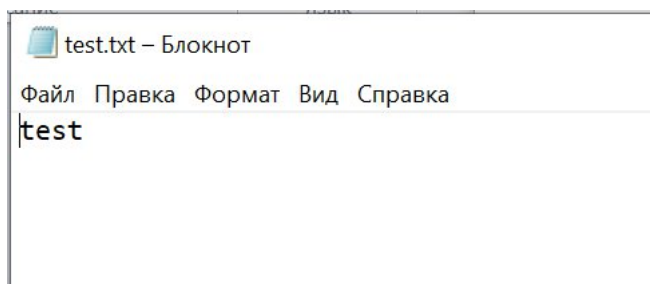
User@S_BMESH_06 MINGW64 ~
$ cd C:/D/GitRepo/tmp

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ git log
commit cf4caddc590903bb220d3c188c1d3c95a72ed9bb (HEAD -> master)
Author: fisa <anfisasunny@gmail.com>
Date: Thu Jan 15 11:39:51 2026 +0300

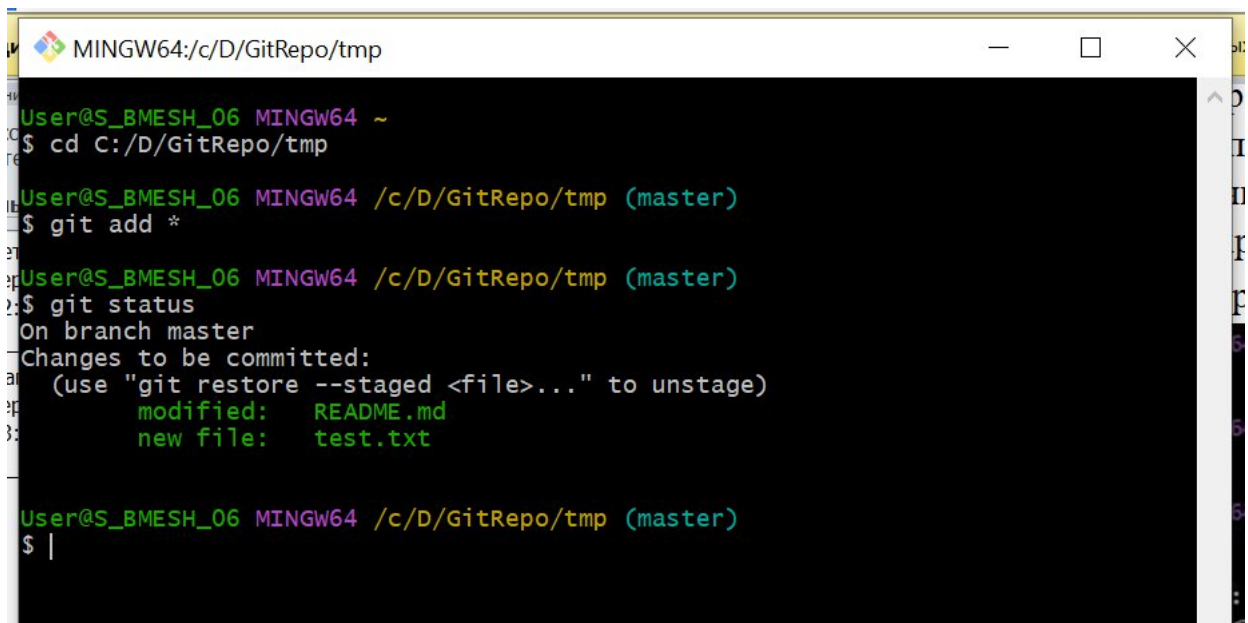
    Добавили новый файл README.md

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ |
```

Создаем новый текстовый файл в папке tmp “test.txt” пишем в нем слово TEST



Добавляем файлы под контроль гита



```
MINGW64:/c/D/GitRepo/tmp

User@S_BMESH_06 MINGW64 ~
$ cd C:/D/GitRepo/tmp

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ git add *

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
        new file:   test.txt

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ |
```

Фиксируем изменения (коммитим)


```

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ git commit -m "тестовый коммит с 2 изменениями"
[master d34241f] тестовый коммит с 2 изменениями
2 files changed, 2 insertions(+)
create mode 100644 test.txt

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$

```

Смотрим историю коммитов

```

MINGW64:/c/D/GitRepo/tmp

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ git commit -m "тестовый коммит с 2 изменениями"
[master d34241f] тестовый коммит с 2 изменениями
2 files changed, 2 insertions(+)
create mode 100644 test.txt

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ git log
commit d34241fdf3988db62a78504210d860f01cb657d0 (HEAD -> master)
Author: fisa <anfisasunny@gmail.com>
Date: Thu Jan 22 11:29:45 2026 +0300

    тестовый коммит с 2 изменениями

commit cf4caddc590903bb220d3c188c1d3c95a72ed9bb
Author: fisa <anfisasunny@gmail.com>
Date: Thu Jan 15 11:39:51 2026 +0300

    Добавили новый файл README.md

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$

```

Изменяем содержимое файла README.md и удаляем файл test.txt

Просматриваем статус репозитория

```

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md
        deleted:    test.txt

no changes added to commit (use "git add" and/or "git commit -a")

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$

```

Чтобы уж точно все сработало добавляем под контроль гита текущие измененные файлы


```

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ git add *

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    test.txt

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ |

```

Git reset HEAD – возвращает в состояние до индексации

```

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ git reset HEAD
Unstaged changes after reset:
M      README.md
D      test.txt

```

Git checkout “name file” - возвращаем в состояние до изменений (поэтапно)

```

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ git checkout *
error: pathspec 'test' did not match any file(s) known to git

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ git checkout test.txt
Updated 1 path from the index

```

Добавляем опять изменений (удаляем опять файл тест, и меняем рэдми)

Git reset –hard HEAD – откат до коммита

```

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ git reset --hard HEAD
warning: unable to rmdir 'test': Directory not empty
HEAD is now at d34241f тестовый коммит с 2 изменениями

```

Пробуем откатиться не на последний коммит а на какой то другой

Для этого через Git log смотрим индефикатор коммита и выполняем git reset –hard «индефикатор коммита»

```

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ git log
commit d34241fdf3988db62a78504210d860f01cb657d0 (HEAD -> master)
Author: fisa <anfisasunny@gmail.com>
Date: Thu Jan 22 11:29:45 2026 +0300

    тестовый коммит с 2 изменениями

commit cf4caddc590903bb220d3c188c1d3c95a72ed9bb
Author: fisa <anfisasunny@gmail.com>
Date: Thu Jan 15 11:39:51 2026 +0300

    Добавили новый файл README.md

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ ^C

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp (master)
$ git reset --hard d34241fdf3988db62a78504210d860f01cb657d0
warning: unable to rmdir 'test': Directory not empty
HEAD is now at d34241f тестовый коммит с 2 изменениями

```

Самостоятельная работа:

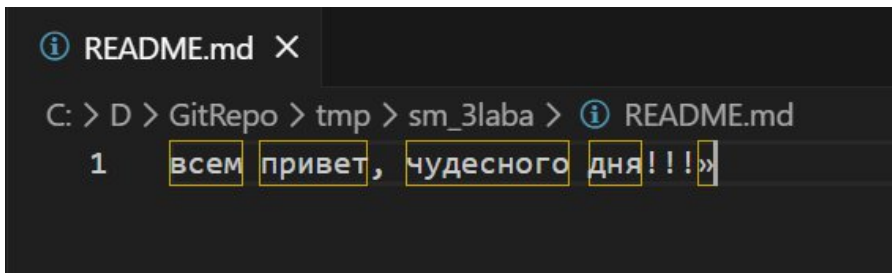
Создаем папку для репозитория, переходим в эту папку через команду `cd` и инициализируем репозиторий.

```

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp/sm_3laba (master)
$ git init
Initialized empty Git repository in C:/D/GitRepo/tmp/sm_3laba/.git/

```

Добавляем в папку файл `README.md` и пишем в нем «всем привет, чудесного дня!!!»



The screenshot shows a Windows File Explorer window with the address bar displaying the path `C: > D > GitRepo > tmp > sm_3laba > README.md`. The file name is `README.md`. The content of the file is displayed as `1 всем привет, чудесного дня!!!`.

Выполняем команду `git status` для просмотра текущего статуса проекта, видим что файл находится в неотслеживаемом состоянии, для того чтобы он изменил свое состояние до отслеживаемого изменяемого выполняем команду `git add *`

```

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp/sm_3laba (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  README.md

nothing added to commit but untracked files present (use "git add" to track)

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp/sm_3laba (master)
$ git *
git: 'README.md' is not a git command. See 'git --help'.

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp/sm_3laba (master)
$ git add *

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp/sm_3laba (master)
$ git status
On branch master

No commits yet

```

Сохраняем изменения через команду `git commit -m "README.md"`

```

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp/sm_3laba (master)
$ git commit -m "README"
[master (root-commit) 0fbfda7] README
 1 file changed, 1 insertion(+)
 create mode 100644 README.md

```

КОММИТИМ ПО ЗАДАНИЮ

```

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp/sm_3laba (master)
$ git commit -m "добавление 3 файлов"
[master c54faf5] добавление 3 файлов
 3 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 1.txt
 create mode 100644 2.txt
 create mode 100644 3.txt

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp/sm_3laba (master)
$ git status
On branch master
nothing to commit, working tree clean

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp/sm_3laba (master)
$ git add *

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp/sm_3laba (master)
$ cit status
bash: cit: command not found

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp/sm_3laba (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   1.txt
    modified:   2.txt

```



```
commit 22671923c007940658722e5df425ad5188cb4a8c
Author: fisa <anfisasunny@gmail.com>
Date: Wed Jan 28 16:41:55 2026 +0300

изменение 1, удаление 2, и добавление еще одного

commit e2d65ebd6a23e6e0b17f138862503b6e4ed8d33e
Author: fisa <anfisasunny@gmail.com>
Date: Wed Jan 28 16:39:40 2026 +0300

изменение 2 файлов

commit baf54ef451c4ec23564afed525efc5e6e1d92b54
Author: fisa <anfisasunny@gmail.com>
Date: Wed Jan 28 16:37:09 2026 +0300

добавление 3 файлов

:|
```

```
User@ MINGW64 /d/GitRepo/sm_3laba (master)
$ mkdir подпвка

User@ MINGW64 /d/GitRepo/sm_3laba (master)
$ git add подпвка

User@ MINGW64 /d/GitRepo/sm_3laba (master)
$ git commit -m "добавление файлов в подпапку"
[master 46f7757] добавление файлов в подпапку
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 "\320\277\320\276\320\264\320\277\320\262\320\277\320\272\32
0\260\Text Document (2).txt"
 create mode 100644 "\320\277\320\276\320\264\320\277\320\262\320\277\320\272\32
0\260\Text Document.txt"
```

```
User@ MINGW64 /d/GitRepo/sm_3laba (master)
$ rm подпанка/*

User@ MINGW64 /d/GitRepo/sm_3laba (master)
$ git add *

User@ MINGW64 /d/GitRepo/sm_3laba (master)
$ git status
On branch master
changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    2.txt
        deleted:    "\\320\\235\\320\\276\\320\\262\\320\\260\\321\\217\\320\\277\\320\\260\\320\\277\\320\\272\\320\\260\\Text Document (2).txt"
        deleted:    "\\320\\235\\320\\276\\320\\262\\320\\260\\321\\217\\320\\277\\320\\260\\320\\277\\320\\272\\320\\260\\Text Document (3).txt"
        deleted:    "\\320\\235\\320\\276\\320\\262\\320\\260\\321\\217\\320\\277\\320\\260\\320\\277\\320\\272\\320\\260\\Text Document.txt"
        deleted:    "\\320\\235\\320\\276\\320\\262\\320\\260\\321\\217\\320\\277\\320\\260\\320\\277\\320\\272\\320\\260\\Text Document (1).txt"
```

Через команду `rmdir` подпапка удаляем подпапку

```

User@ MINGW64 /d/GitRepo/sm_31aba (master)
$ rmdir подпапка

User@ MINGW64 /d/GitRepo/sm_31aba (master)
$ пше фвв *
bash: пше: command not found

User@ MINGW64 /d/GitRepo/sm_31aba (master)
$ git add *

```

Основные заполнители (часто используемые)

%H — полный хэш коммита;

%h — сокращённый хэш (7 символов);

%an — имя автора;

%ae — email автора;

%cn — имя коммиттера (того, кто закоммитил);

%ce — email коммиттера;

%ad — дата автора (в формате репо);

%cd — дата коммиттера (в формате репо);

%ar — относительная дата (например, «2 дня назад»);

%s — сообщение коммита (subject);

%b — тело сообщения коммита (body);

%d — референсы (ветки, теги) для коммита;

%n — перевод строки;

%C(цвет) — смена цвета вывода (см. ниже).

Командой нже я вывела кратки однострочный список коммитов

```

User@ MINGW64 /d/GitRepo/sm_31aba (master)
$ git log --pretty=format:"%h - %s"
d8b1fb3 - 5
46f7757 - добавление файлов в подпапку
0e06061 - создание папки и добавление в нее файлов
3218faf - 3 пункт самостоятельной
2267192 - изменение 1, удаление 2, и добавление еще одного
e2d65eb - изменение 2 файлов
baf54ef - добавление 3 файлов

```

Контрольные вопросы:

1. **Как отобразить текущий статус репозитория?** Текущий статус репозитория отображается с помощью команды `GIT STATUS`.
2. **Какие файлы называются Untracked files?** Untracked files — это файлы, которые ещё не добавлены в индекс Git и не отслеживаются системой контроля версий.
3. **Как передать файл под контроль Git?** Чтобы передать файл под контроль Git, нужно выполнить команду:
`GIT ADD <FILE>`— для конкретного файла;
`GIT ADD .` — для всех файлов в директории.
4. **Как отобразить список изменений в репозитории?** Для отображения списка изменений используются команды:
`GIT DIFF`— показывает текущие изменения;
`GIT LOG`— отображает историю коммитов.
5. **Как отменить коммит и почему это делать не рекомендуется?** Чтобы отменить коммит, используют команду `GIT RESET HEAD ~ 1` (откатывает последний коммит). Это не рекомендуется, так как нарушает историю проекта.
6. **Как Git идентифицирует все этапы изменений?** Git идентифицирует изменения по уникальным хешам (ID) для каждого коммита и состояния файлов

Лабораторная работа №4

```
User@ MINGW64 /d/GitRepo
$ git clone https://github.com/fisyarik/fish228.git
Cloning into 'fish228'...
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 9 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (9/9), done.
```

```
User@S_BMESH_05 MINGW64 /c/D/GitRepo/tmp (master)
$ cd GitTest

User@S_BMESH_05 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
$ git add hi.txt

User@S_BMESH_05 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   hi.txt
```

```
User@S_BMESH_05 MINGW64 /c/D/GitRepo/tmp (master)
$ cd GitTest

User@S_BMESH_05 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
$ git add hi.txt

User@S_BMESH_05 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   hi.txt

User@S_BMESH_05 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
$ git remote -v
origin https://github.com/kozlovalova/GitTest.git (fetch)
origin https://github.com/kozlovalova/GitTest.git (push)

User@S_BMESH_05 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   hi.txt
```

```
User@S_BMESH_05 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
$ git commit -m "Hi.txt"
[main a77ee20] Hi.txt
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 hi.txt
```

```
User@S_BMESH_05 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
$ git push
info: please complete authentication in your browser...
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 270 bytes | 67.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/kozlovalova/GitTest.git
d52b5c1..a77ee20 main -> main
```

```
User@ MINGW64 /d/GitRepo/fish228 (main)
$ git log
commit 8d32112b3a4d4d246ebf2728f0675132266c8e19 (HEAD -> main, origin/main, origin/HEAD)
Author: fisyarik <anfisasunny@gmail.com>
Date: Thu Jan 22 11:51:40 2026 +0300

    Update README.md

commit a66fab32222af2afc81a071e5665e990116d4e05
Author: fisa <anfisasunny@gmail.com>
Date: Thu Jan 22 11:47:06 2026 +0300

    hi.txt

commit f214fa3d7296a2cb471f291d0d180ff5a7b0c077
Author: fisyarik <anfisasunny@gmail.com>
Date: Thu Jan 22 11:08:51 2026 +0300

    Initial commit
```

Cat – для того , чтобы смотреть

git pull

git push

Git Remote -v – команда для отвязки от одного репозитория к другому

Самостоятельная работа:

Создаем репозиторий на компьютере и инициализируем его

```
User@ MINGW64 /d/GitRepo
$ mkdir fish2_228

User@ MINGW64 /d/GitRepo
$

User@ MINGW64 /d/GitRepo
$ cd D:/GitRepo/fish2_228

User@ MINGW64 /d/GitRepo/fish2_228
$ git init
Initialized empty Git repository in D:/GitRepo/fish2_228/.git/
```

Связываем его с первым репозиторием и проверяем

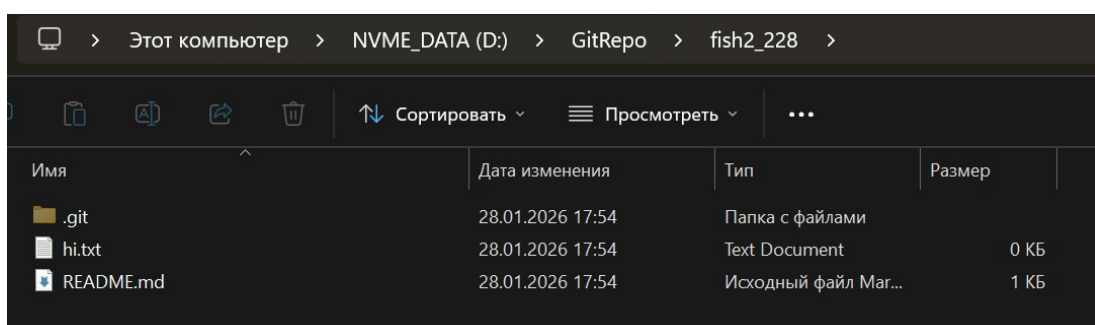
```
User@ MINGW64 /d/GitRepo/fish2_228 (master)
$ git remote add origin https://github.com/fisyarik/fish228.git

User@ MINGW64 /d/GitRepo/fish2_228 (master)
$ git remote -v
origin https://github.com/fisyarik/fish228.git (fetch)
origin https://github.com/fisyarik/fish228.git (push)
```

Скачиваем данные из гитхаба

```
User@ MINGW64 /d/GitRepo/fish2_228 (master)
$ git fetch origin
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 9 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (9/9), 2.00 KiB | 93.00 KiB/s, done.
From https://github.com/fisyarik/fish228
* [new branch]      main      -> origin/main
```

И видим что все данные из первого репозитория перенеслись во второй



Имя	Дата изменения	Тип	Размер
.git	28.01.2026 17:54	Папка с файлами	
hi.txt	28.01.2026 17:54	Text Document	0 КБ
README.md	28.01.2026 17:54	Исходный файл Маг...	1 КБ

Создаем репозиторий А в гитхабе и добавлем в качестве origin для первого репозитория , предварительно удалив предыдущий origin

```
User@ MINGW64 /d/GitRepo/fish228 (main)
$ git remote remove origin
```

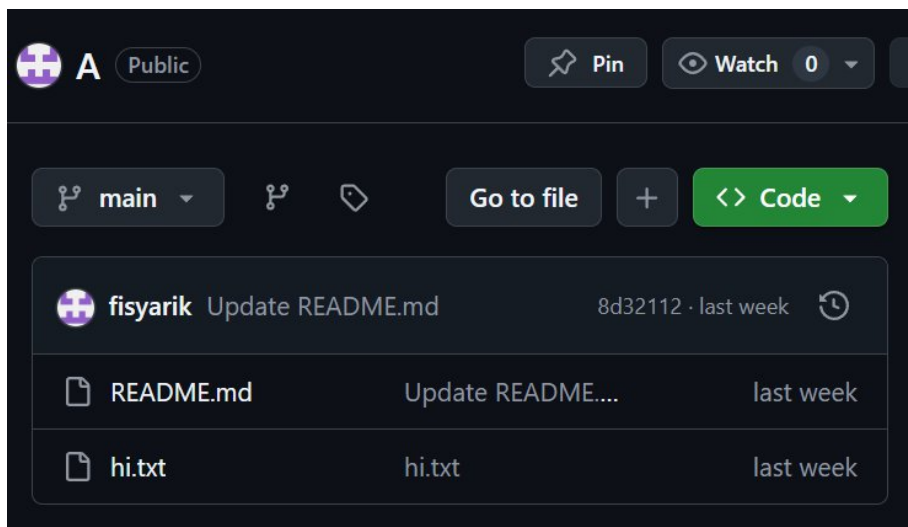
```
User@ MINGW64 /d/GitRepo/fish228 (main)
$ git remote add origin https://github.com/fisyarik/A.git
```

Отправляем данные в репозиторий А и клонируем его себе

```
User@ MINGW64 /d/GitRepo/fish228 (main)
$ git push -u origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 2.02 KiB | 2.02 MiB/s, done.
Total 9 (delta 0), reused 9 (delta 0), pack-reused 0 (from 0)
To https://github.com/fisyarik/A.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

User@ MINGW64 /d/GitRepo/fish228 (main)
$
```

Проверяем



Ссылка на репозиторий А <https://github.com/fisyarik/A.git>

Создаем репозиторий В в гитхабе и клонируем его себе

```
User@ MINGW64 /d/GitRepo
$ git clone https://github.com/fisyarik/B.git
Cloning into 'B'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

Делаем несколько коммитов и пушим в гитхаб

```
User@ MINGW64 /d/GitRepo/B (main)
$ git add .

User@ MINGW64 /d/GitRepo/B (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
```

```
User@ MINGW64 /d/GitRepo/B (main)
$ git commit -m "изменение рэдми"
[main d1f43ef] изменение рэдми
1 file changed, 3 insertions(+), 1 deletion(-)
```

```
User@ MINGW64 /d/GitRepo/B (main)
$ git add .
```

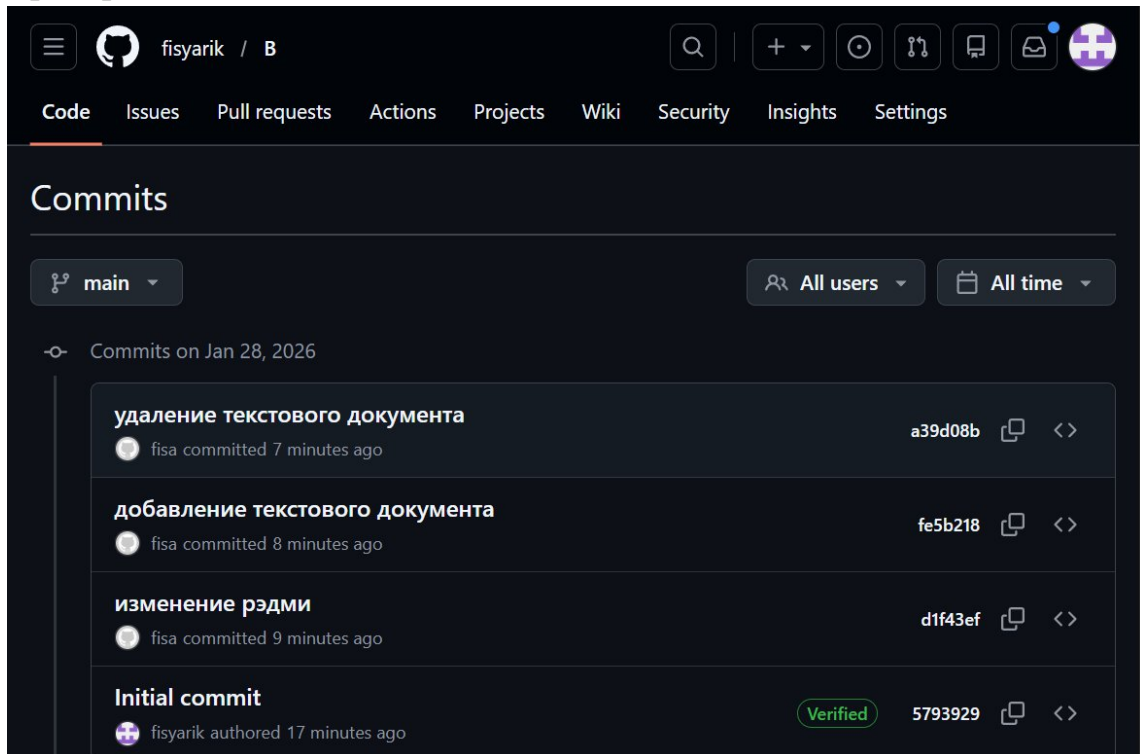
```
User@ MINGW64 /d/GitRepo/B (main)
$ git commit -m "добавление текстового документа"
[main fe5b218] добавление текстового документа
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Text Document.txt
```

```
User@ MINGW64 /d/GitRepo/B (main)
$ git add .
```

```
User@ MINGW64 /d/GitRepo/B (main)
$ git commit -m "удаление текстового документа"
[main a39d08b] удаление текстового документа
1 file changed, 0 insertions(+), 0 deletions(-)
delete mode 100644 Text Document.txt
```

```
User@ MINGW64 /d/GitRepo/B (main)
$ git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (7/7), 793 bytes | 396.00 KiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/fisyarik/B.git
5793929..a39d08b  main -> main
```


Проверяем в гит хабе коммиты



Ссылка на репозиторий <https://github.com/fisyarik/B.git>

Контрольные вопросы:

1. **Как отправить изменения на удалённый репозиторий?** Изменения отправляются на удалённый репозиторий с помощью команды `GIT PUSH ORIGIN <ВЕТКА>`.
2. **Как влить изменения в локальную ветку из удалённого репозитория?** Для вливания изменений из удалённого репозитория используется команда `GIT PULL`.
3. **Какой командой можно зафиксировать изменения в репозитории?** Изменения фиксируются командой `GIT COMMIT –М “КОМЕНТАРИЙ”`.
4. **Как используется GitHub в реальной работе?** GitHub используется для:
 - хостинга кода;
 - управления задачами;
 - код-ревью;
 - CI/CD (непрерывная интеграция и доставка);
 - коллаборации (совместной работы) над проектами.
- **Опишите принцип коллективной работы с GitHub** Принцип коллективной работы с GitHub:
 - Форк репозитория.

- Клонирование репозитория на локальный компьютер.
- Внесение изменений.
- Коммит изменений.
- Пуш изменений в свой форк.
- Создание Pull Request (PR) для слияния изменений в основной репозиторий.

Лабораторная работа №5

Работа с удаленными репозиториями

```

MINGW64/c:/D/GitRepo/tmp2
User@S_BMESH_06 MINGW64 ~
$ cd C:/D/GitRepo/tmp2
User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp2
$ git init
Initialized empty Git repository in C:/D/GitRepo/tmp2/.git/
User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp2 (master)
$
User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp2 (master)
$ |

MINGW64/c:/D/GitRepo2
User@S_BMESH_06 MINGW64 /c/D/GitRepo2
$ git init
Initialized empty Git repository in C:/D/GitRepo2/.git/
User@S_BMESH_06 MINGW64 /c/D/GitRepo2 (master)
$ |
  
```

```

User@S_BMESH_06 MINGW64 /c/D/GitRepo2 (master)
$ git add .

User@S_BMESH_06 MINGW64 /c/D/GitRepo2 (master)
$ git commit -m "readme"
[master (root-commit) b0c3363] readme
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 README.md
  
```

Git remote add название паки и путь папки – для связи репозитория

```

User@S_BMESH_06 MINGW64 /c/D/GitRepo2 (master)
$ git remote add hihik "C:\D\GitRepo2"
  
```

Git remote – выводит связи

```

User@S_BMESH_06 MINGW64 /c/D/GitRepo2 (master)
$ git remote add hihik "C:\D\GitRepo2"

User@S_BMESH_06 MINGW64 /c/D/GitRepo2 (master)
$ git remote -v
hihik    C:\D\GitRepo2 (fetch)
hihik    C:\D\GitRepo2 (push)
  
```

Git fetch имя папки – переносит данные из заданой папки

Git checkout –track имя папки/имя ветки – связь текущей ветки с веткой которую указали

```

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp2 (master)
$ git fetch hihik
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 188 bytes | 15.00 KiB/s, done.
From C:/D/Repo2
   * [new branch]      master      -> hihik/master

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp2 (master)
$ git checkout --track hihik/master
branch 'master' set up to track 'hihik/master'.
Already on 'master'

```

Получаем изменения

```

User@S_BMESH_06 MINGW64 /c/D/GitRepo/tmp2 (master)
$ git pull
Already up to date.

```

Контрольные вопросы:

1. **Что такое ветка?** Ветка (branch) — это независимая линия разработки в репозитории, позволяющая работать над разными задачами параллельно.
2. **На какие типы делятся репозитории Git?** Репозитории Git делятся на два типа:
 - локальные (на ПК пользователя);
 - удалённые (на серверах, например, GitHub).
3. **Как добавить ссылку на удалённый репозиторий?** Ссылка на удалённый репозиторий добавляется командой `GIT REMOTE ADD ORIGIN`.
4. **Как вывести подробный список прикреплённых репозиториях?** Подробный список прикреплённых репозиториях выводится командой `GIT REMOTE -V`.
5. **Что выполняет команда `git remote show`?** Команда `GIT REMOTE SHOW` выводит детали о удалённом репозитории, включая:
 - ветки;
 - теги;
 - URL-адрес репозитория.

Лабораторная работа №6

Работа с метками

```
User@S_BMESH_06 MINGW64 /c/D/GitRepo/branches (master)
$ git add .

User@S_BMESH_06 MINGW64 /c/D/GitRepo/branches (master)
$ git commit -m "document"
[master (root-commit) 3e7d18e] document
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 "\320\235\320\276\320\262\321\213\320\271 \321\202\320\265\320\272\321\201\321\202\320\276\320\262\321\213\320\271 \320\264\320\276\320\272\321\203\320\274\320\265\320\275\321\202.txt"

User@S_BMESH_06 MINGW64 /c/D/GitRepo/branches (master)
$ git tag 1.0

User@S_BMESH_06 MINGW64 /c/D/GitRepo/branches (master)
$ git tag
1.0
```

```
User@S_BMESH_06 MINGW64 /c/D/GitRepo/branches (master)
$ git show 1.0
commit 3e7d18ed837fe1b641031600b8d7acae9b9125b8 (HEAD -> master, tag: 1.0)
Author: fisa <anfisasunny@gmail.com>
Date: Thu Jan 29 11:46:15 2026 +0300

    document

diff --git "a/\320\235\320\276\320\262\321\213\320\271 \321\202\320\265\320\272\321\201\321\202\320\276\320\262\321\213\320\271 \320\264\320\276\320\272\321\203\320\274\320\265\320\275\321\202.txt" "b/\320\235\320\276\320\262\321\213\320\271 \321\202\320\265\320\272\321\201\321\202\320\276\320\262\321\213\320\271 \320\264\320\276\320\272\321\203\320\274\320\265\320\275\321\202.txt"
new file mode 100644
index 0000000..e69de29
```

Просмотр списка коммитов

```
User@S_BMESH_06 MINGW64 /c/D/GitRepo/branches (master)
$ git log --oneline
3e7d18e (HEAD -> master, tag: 1.0) document
```

Создание тега 1.1 для старого коммита

```
User@S_BMESH_06 MINGW64 /c/D/GitRepo/branches (master)
$ git tag 1.2 3e7d18e
```

Контрольные вопросы:

1. **Что такое метка?** Метка (tag) — это ссылка на конкретный коммит, используемая для обозначения версии проекта.
2. **Какие типы меток существуют?** Существуют два типа меток:
 - лёгкие (lightweight) — только ссылка на коммит;
 - аннотированные (annotated) — содержат комментарий и метаданные.

3. **Как добавляется метка к репозиторию?** Метка добавляется командой `GIT TAG`
-А <ИМЯ МЕТКИ>-М “КОММЕНТАРИЙ”
4. **Как вывести информацию по метке?** Информация по метке выводится командой `GIT SHOW <ИМЯ МЕТКИ>`-
5. **Для чего нужен сабмодуль?** Сабмодуль — это встроенный репозиторий внутри другого репозитория, используемый для работы с библиотеками и зависимостями.
6. **Как добавить сабмодуль?** Сабмодуль добавляется командой `GIT SUBMODULE ADD <URL РЕПОЗИТОРИЯ>`

Лабораторная работа №7

Создаем папку для репозитория

```
User@S_BMESH_06 MINGW64 ~  
$ cd C:\D  
  
User@S_BMESH_06 MINGW64 /c/D  
$ mkdir ProGit  
  
User@S_BMESH_06 MINGW64 /c/D  
$ mkdir C:/D/ProGit/branches
```

Инициализируем репозиторий

```
User@S_BMESH_06 MINGW64 /c/D  
$ cd C:/D/ProGit/branches  
  
User@S_BMESH_06 MINGW64 /c/D/ProGit/branches  
$ git init  
Initialized empty Git repository in C:/D/ProGit/branches/.git/
```

Проверяем есть ли у нас какие то коммиты и какая ветка

```
User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (master)  
$ git status  
On branch master  
  
No commits yet  
  
nothing to commit (create/copy files and use "git add" to track)
```

Создаем файл Readme

И добавляем ее в контроль git и коммитим

```
User@S_BMESH_06 MINGW64 /c/D/Progit/branches (master)
$ git add .

User@S_BMESH_06 MINGW64 /c/D/Progit/branches (master)
$ git commit -m "first commit"
[master (root-commit) 908c944] first commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 README.md
```

Проверяем статус репозитория и в какой ветке находимся

```
User@S_BMESH_06 MINGW64 /c/D/Progit/branches (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Смотрим список коммитов

```
User@S_BMESH_06 MINGW64 /c/D/Progit/branches (master)
$ git log
commit 908c9443cba4500e90e4c3627cb65850aeea057f (HEAD -> master)
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 10:52:31 2026 +0300

    first commit
```

Создаем новую ветку

```
User@S_BMESH_06 MINGW64 /c/D/Progit/branches (master)
$ git branch testing

User@S_BMESH_06 MINGW64 /c/D/Progit/branches (master)
$ git status
On branch master
nothing to commit, working tree clean
```

И переходим в эту ветку

```
User@S_BMESH_06 MINGW64 /c/D/Progit/branches (master)
$ git checkout testing
Switched to branch 'testing'

User@S_BMESH_06 MINGW64 /c/D/Progit/branches (testing)
$ git status
On branch testing
nothing to commit, working tree clean
```

Меняем файл README и коммитим в ветке testing

```
User@S_BMESH_06 MINGW64 /c/D/Progit/branches (testing)
$ git add .

User@S_BMESH_06 MINGW64 /c/D/Progit/branches (testing)
$ git commit -m "КОММИТ В ВЕТКЕ testing"
[testing 2ff088a] КОММИТ В ВЕТКЕ testing
1 file changed, 1 insertion(+)
```

Проверяем коммиты и видим что коммиты находятся в разных ветках

```
User@S_BMESH_06 MINGW64 /c/D/Progit/branches (testing)
$ git log
commit 2ff088a95169b0fb6b6a10b3ed4fcf790dd69249 (HEAD -> testing)
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 11:00:34 2026 +0300

    КОММИТ В ВЕТКЕ testing

commit 908c9443cba4500e90e4c3627cb65850aeea057f (master)
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 10:52:31 2026 +0300

    first commit
```

Переходим в ветку мастер и смотрим коммиты

```
User@S_BMESH_06 MINGW64 /c/D/Progit/branches (testing)
$ git checkout master
Switched to branch 'master'

User@S_BMESH_06 MINGW64 /c/D/Progit/branches (master)
$ git log
commit 908c9443cba4500e90e4c3627cb65850aeea057f (HEAD -> master)
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 10:52:31 2026 +0300

    first commit
```

Видим что отображаются коммиты только которые были сделаны в этой ветке

Добавляем новый файл и передаем под контроль гита в ветке master

```
User@S_BMESH_06 MINGW64 /c/D/Progit/branches (master)
$ git add .
```

Коммитим и смотрим список коммитов, видим что добавился новый коммит


```

User@S_BMESH_06 MINGW64 /c/D/Progit/branches (master)
$ git commit -m "добавление index.txt в ветке master"
[master ea61917] добавление index.txt в ветке master
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 index.txt

User@S_BMESH_06 MINGW64 /c/D/Progit/branches (master)
$ git log
commit ea619170e91c96b95c913d0159379e3f76f1850b (HEAD -> master)
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 11:13:12 2026 +0300

    добавление index.txt в ветке master

commit 908c9443cba4500e90e4c3627cb65850aeea057f
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 10:52:31 2026 +0300

    first commit

```

Переходим в ветку testing и видим что ничего не поменялось в этой ветке, так и осталось 2 коммита

```

User@S_BMESH_06 MINGW64 /c/D/Progit/branches (master)
$ git checkout testing
Switched to branch 'testing'
g
User@S_BMESH_06 MINGW64 /c/D/Progit/branches (testing)
$ git log
commit 2ff088a95169b0fb6b6a10b3ed4fcf790dd69249 (HEAD -> testing)
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 11:00:34 2026 +0300

    КОММИТ В ВЕТКЕ testing

commit 908c9443cba4500e90e4c3627cb65850aeea057f
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 10:52:31 2026 +0300

    first commit

```

Создаем новую ветку перед этим перейдя в ветку master

```

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (testing)
$ git checkout master
Switched to branch 'master'

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (master)
$ git branch hotfix

```

Переходим в эту ветку

```

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (master)
$ git checkout hotfix
Switched to branch 'hotfix'

```


Меняем файл index и смотрим статус

```
User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (hotfix)
$ git status
On branch hotfix
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.txt
```

Добавляем изменения под контроль гита и коммитим

```
User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (hotfix)
$ git add .

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (hotfix)
$ git commit -m "Добавили ркасную кнопку"
[hotfix c8ce1d0] Добавили ркасную кнопку
1 file changed, 2 insertions(+)
```

Список коммитов и видим что в нашей новой ветке сохранились все коммиты из master и новый наш

```
User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (hotfix)
$ git log
commit c8ce1d07fd3b6ede5153a28f36a8b6f2522304a0 (HEAD -> hotfix)
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 15:25:58 2026 +0300

    Добавили ркасную кнопку

commit c90c27fd586f00e2cbdd00f17a458123b01617b5 (master)
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 15:16:46 2026 +0300

    commit in master

commit 102b386e35e5fc2f7dbbb915ab59436e6ab4c9
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 15:08:43 2026 +0300

    first commit
```

Переходим в ветку Master и выводим список коммитов, видим что прошлый коммит который был создан в новой ветке не отображается

```

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (hotfix)
$ git checkout master
Switched to branch 'master'

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (master)
$ git log
commit c90c27fd586f00e2cbdd00f17a458123b01617b5 (HEAD -> master)
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 15:16:46 2026 +0300

    commit in master

commit 102b386e35e5fc2f7dbbb915ab59436e6ab4c9
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 15:08:43 2026 +0300

    first commit

```

Объединяем с доп веткой

```

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (master)
$ git merge hotfix
Updating c90c27f..c8ce1d0
Fast-forward
 index.txt | 2 ++
 1 file changed, 2 insertions(+)

```

Переходим в папку и смотрим что индекс изменился, а до этого он был без дополнительной надписи которую мы коммитили в другой ветке с которой объединили

Смотрим список коммитов и видим что теперь 3 коммита и один из них после объединения двух веток

```

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (master)
$ git log
commit c8ce1d07fd3b6ede5153a28f36a8b6f2522304a0 (HEAD -> master, hotfix)
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 15:25:58 2026 +0300

    Добавили красную кнопку

commit c90c27fd586f00e2cbdd00f17a458123b01617b5
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 15:16:46 2026 +0300

    commit in master

commit 102b386e35e5fc2f7dbbb915ab59436e6ab4c9
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 15:08:43 2026 +0300

    first commit

```

Переходим в ветку `testing` и просматриваем коммиты

```
User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (master)
$ git checkout testing
Switched to branch 'testing'

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (testing)
$ git log
commit 6a618319607cbcac19c68f85c4528afb591d77ce (HEAD -> testing)
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 15:11:01 2026 +0300

    commit in testing

commit 102b386e35e5fc2f7dbbb915ab59436e6ab4c9
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 15:08:43 2026 +0300

    first commit
```

Переходим обратно в ветку master и сливаем с веткой testing

```
User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (testing)
$ git checkout master
Switched to branch 'master'

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (master)
$ git merge testing
```

После чего открывается текстовый редактор

```
Merge branch 'testing'
# Please enter a commit message to explain why this merge is necessary,
# especially if it merges an updated upstream into a topic branch.
#
# Lines starting with '#' will be ignored, and an empty message aborts
# the commit.
```

```
~
~
~
~
~
~
~
~
~
~
~
~
~
```

```
.git/MERGE_MSG [unix] (15:50 05/02/2026) 1,1 All  
"/c/D/ProGit/branches/.git/MERGE_MSG" [unix] 6L, 249B
```


Вернулись обратно в git bash через последовательное нажатие <esc> : q

```
Merge made by the 'ort' strategy.
 README.md | 1 +
 1 file changed, 1 insertion(+)

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (master)
$
```

Вводим команду git log и видим созданный коммит который мы могли переименовать в текстовом редакторе но мы этого не делали

```
commit 2bdacd5ec60dba2de2a3cc9a83be8a9c2839d6d6 (HEAD -> master)
Merge: c8ce1d0 6a61831
Author: fisa <anfisasunny@gmail.com>
Date: Thu Feb 5 15:50:08 2026 +0300

    Merge branch 'testing'
```

Коммитим

```
User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (master)
$ git add .

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (master)
$ git commit -m "создали файл"
[master 454a05a] создали файл
 1 file changed, 1 insertion(+)
 create mode 100644 first.txt
```

Создаем ветку и переходим в нее

```
User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (master)
$ git checkout -b feature/1
Switched to a new branch 'feature/1'

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (feature/1)
$ |
```

Мы поменяли текстовый файл добавили новую надпись и закоммитили изменения

```
User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (feature/1)
$ git add .

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (feature/1)
$ git commit -m "добавили красную кнопку"
[feature/1 0e586ed] добавили красную кнопку
 1 file changed, 1 insertion(+)
```


Перешли обратно в ветку мастер и создали вторую ветку и перешли в нее

```
User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (feature/1)
$ git checkout master
Switched to branch 'master'

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (master)
$ git branch feature/2

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (master)
$ git checkout feature/2
Switched to branch 'feature/2'
```

Добавили надпись две зеленых кнопки и закоммитили изменения

```
User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (feature/2)
$ git add .

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (feature/2)
$ git commit -m "две зеленых кнопки"
[feature/2 17bd8ea] две зеленых кнопки
1 file changed, 1 insertion(+)
```

Переходим в первую ветку где красная кнопка объединяем ее со второю где зеленые

```
User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (feature/2)
$ git checkout feature/1
Switched to branch 'feature/1'

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (feature/1)
$ git merge feature/2
Auto-merging first.txt
CONFLICT (content): Merge conflict in first.txt
Automatic merge failed; fix conflicts and then commit the result.
```

Мы видим надпись CONFLICT – в файле first.txt возник mergeконфликт. Автоматическое слияние невозможно. Нужно поправить конфликты и закоммитить результат.

Заходим в файл текстовый и видим

```
first.txt – Блокнот
Файл Правка Формат Вид Справка
это первая строка
<<<<<<< HEAD
красная кнопка
=====
тут две зеленых кнопки
>>>>>>> feature/2
```

Удаляем все служебные команды и сохраняем файл смотрим статус репозитория

```
User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (feature/1|MERGING)
$ git status
On branch feature/1
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   first.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

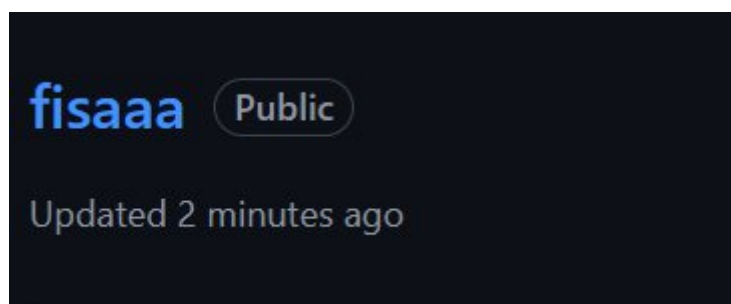
Коммитим изменения

```
User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (feature/1|MERGING)
$ git add .

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (feature/1|MERGING)
$ git commit -m "конфликт решен"
[feature/1 41aebbb] конфликт решен
```

Самостоятельная работа:

Создаем новый пустой репозиторий на гитхабе



Клонируем его в папку D вместо диска D

```
User@ MINGW64 /c/D
$ git clone https://github.com/fisyyarik/fisaaa.git
Cloning into 'fisaaa'...
warning: You appear to have cloned an empty repository.
User@ MINGW64 /c/D
```

Созадем файл HTML с моим профилем

```
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="ru">
3  <head>
4      <meta charset="UTF-8">
5      <title>Мой профиль</title>
6      <link rel="stylesheet" href="style.css">
7  </head>
8  <body>
9      <h1>прелкина фиса</h1>
10     <p>Веб-разработчик. Специализируюсь на C# b Python</p>
11     <p>Александр Сергеевич не убивайте я набыю тату на лбу</p>
12 </body>
13 </html>
14
```

Переходим в папку репозитория

Добавляем добавленный файл по контроль гита

И коммитим изменения

```
User@ MINGW64 /c/D
$ cd C:/D/fisaaa

User@ MINGW64 /c/D/fisaaa (main)
$ git add .

User@ MINGW64 /c/D/fisaaa (main)
$ git commit -m "добавила index.html с базовой информацией обо мне"
[main (root-commit) 58ce778] добавила index.html с базовой информацией обо мне
1 file changed, 13 insertions(+)
create mode 100644 index.html
```


Пушим в гитхаб в ветку мэйн

```
User@ MINGW64 /c/D/fisaaa (main)
$ git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 570 bytes | 285.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/fisyarik/fisaaa.git
* [new branch]      main -> main
```

добавляем ветку и сразу же переходим в нее

```
User@ MINGW64 /c/D/fisaaa (main)
$ git checkout -b add-subscribe-button
Switched to a new branch 'add-subscribe-button'

User@ MINGW64 /c/D/fisaaa (add-subscribe-button)
$ |
```

Добавляем кнопку

```
<p>Александр Сергеевич не убивайте я набью
<button>Подписаться на новости</button>
ody>
```

Добавляем изменения под контроль гит и коммитим

```
User@ MINGW64 /c/D/fisaaa (add-subscribe-button)
$ git add .

User@ MINGW64 /c/D/fisaaa (add-subscribe-button)
$ git commit -m "Добавлена кнопка 'Подписаться на новости'"
[add-subscribe-button 7d868c1] Добавлена кнопка 'Подписаться на новости'
1 file changed, 1 insertion(+)
```

Пушим на гит хаб через ветку созданную для кнопки

```
User@ MINGW64 /c/D/fisaaa (add-subscribe-button)
$ git push origin add-subscribe-button
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 403 bytes | 201.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'add-subscribe-button' on GitHub by visiting:
remote:      https://github.com/fisyarik/fisaaa/pull/new/add-subscribe-button
remote:
To https://github.com/fisyarik/fisaaa.git
* [new branch]      add-subscribe-button -> add-subscribe-button
```


Переходим в ветку main

```
User@ MINGW64 /c/D/fisaaa (add-subscribe-button)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

Выполняем слияние веток

```
User@ MINGW64 /c/D/fisaaa (main)
$ git merge add-subscribe-button
Updating 58ce778..7d868c1
Fast-forward
 index.html | 1 +
 1 file changed, 1 insertion(+)
```

И пушим на гитхаб

```
User@ MINGW64 /c/D/fisaaa (main)
$ git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/fisyarik/fisaaa.git
 58ce778..7d868c1  main -> main
```

Создаем ветку для изменения цвета кнопки

```
User@ MINGW64 /c/D/fisaaa (main)
$ git checkout -b change-button-color
Switched to a new branch 'change-button-color'
```

Изменяем цвет в файле html

```
<button {
background-color: #4CAF50;
color: white;
border: none;
padding: 10px 20px;}>Подписаться на новости</button>
ody>
```

Добавляем изменения под контроль гит и коммитим

```
User@ MINGW64 /c/D/fisaaa (change-button-color)
$ git add .

User@ MINGW64 /c/D/fisaaa (change-button-color)
$ git commit -m "Изменён цвет кнопки на зелёный"
[change-button-color c06b420] Изменён цвет кнопки на зелёный
1 file changed, 5 insertions(+), 1 deletion(-)
```

Пушим на гитхаб

```
User@ MINGW64 /c/D/fisaaa (change-button-color)
$ git push origin change-button-color
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 396 bytes | 396.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'change-button-color' on GitHub by visiting:
remote:   https://github.com/fisyarik/fisaaa/pull/new/change-button-color
remote:
To https://github.com/fisyarik/fisaaa.git
 * [new branch]      change-button-color -> change-button-color

User@ MINGW64 /c/D/fisaaa (change-button-color)
```

Переходим в ветку мэйн

Сливаем ветки

И пушим на гитхаб

```
User@ MINGW64 /c/D/fisaaa (change-button-color)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

User@ MINGW64 /c/D/fisaaa (main)
$ git merge change-button-color
Updating 7d868c1..c06b420
Fast-forward
 index.html | 6 +++++-
 1 file changed, 5 insertions(+), 1 deletion(-)

User@ MINGW64 /c/D/fisaaa (main)
$ git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/fisyarik/fisaaa.git
 7d868c1..c06b420  main -> main
```

Добавляем ветку

```
User@ MINGW64 /c/D/fisaaa (main)
$
git checkout -b add-social-login
Switched to a new branch 'add-social-login'
```

Добавляем форму входа через соцсети

```
padding: 10px 20px;}>Подписаться на н
<div class="social-login">
  <button>Войти через Google</button>
  <button>Войти через VK</button>
</div>
```

Добавляем изменения под контроль гит и коммитим

```
User@ MINGW64 /c/D/fisaaa (add-social-login)
$ git add .

User@ MINGW64 /c/D/fisaaa (add-social-login)
$ git commit -m "Добавлена форма входа через соцсети"
[add-social-login df4bd83] Добавлена форма входа через соцсети
1 file changed, 4 insertions(+)
```

Пушим на гитхаб

```
User@ MINGW64 /c/D/fisaaa (add-social-login)
$ git push origin add-social-login
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 425 bytes | 425.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'add-social-login' on GitHub by visiting:
remote:      https://github.com/fisyarik/fisaaa/pull/new/add-social-login
remote:
To https://github.com/fisyarik/fisaaa.git
 * [new branch]      add-social-login -> add-social-login
```


Переходим в ветку main

Сливаем ветки

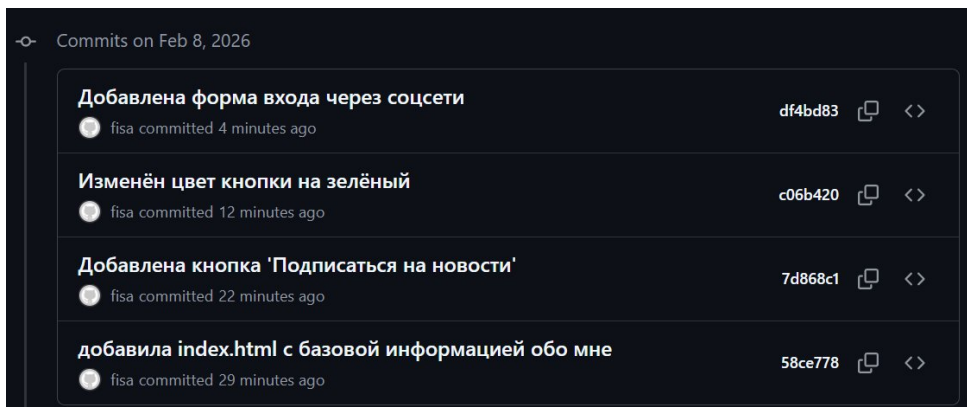
Пушим

```
User@ MINGW64 /c/D/fisaaa (add-social-login)
$
git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

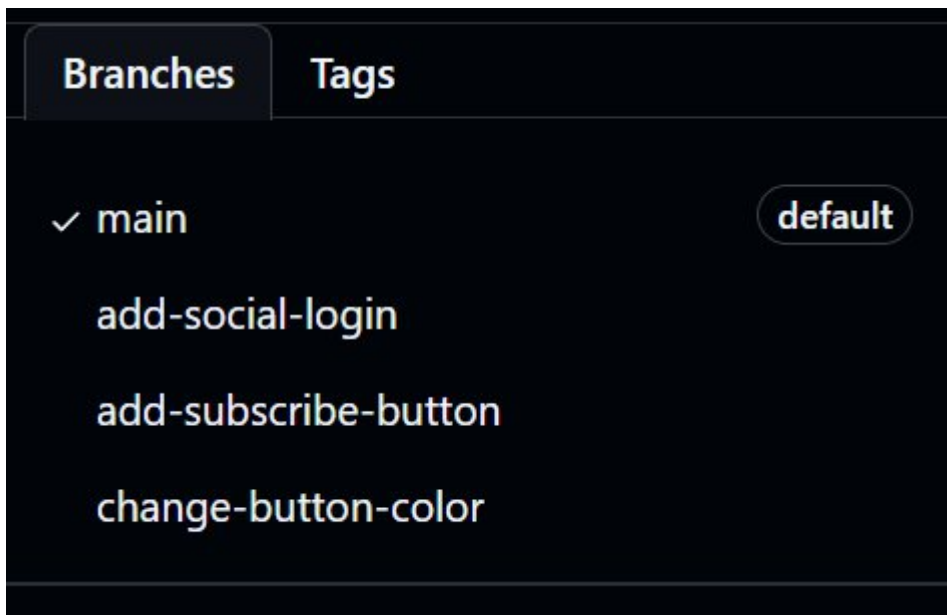
User@ MINGW64 /c/D/fisaaa (main)
$ git merge add-social-login
Updating c06b420..df4bd83
Fast-forward
 index.html | 4 ++++
 1 file changed, 4 insertions(+)

User@ MINGW64 /c/D/fisaaa (main)
$ git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/fisyarik/fisaaa.git
 c06b420..df4bd83  main -> main
```

Проверяем гитхаб Видим что было 4 коммита

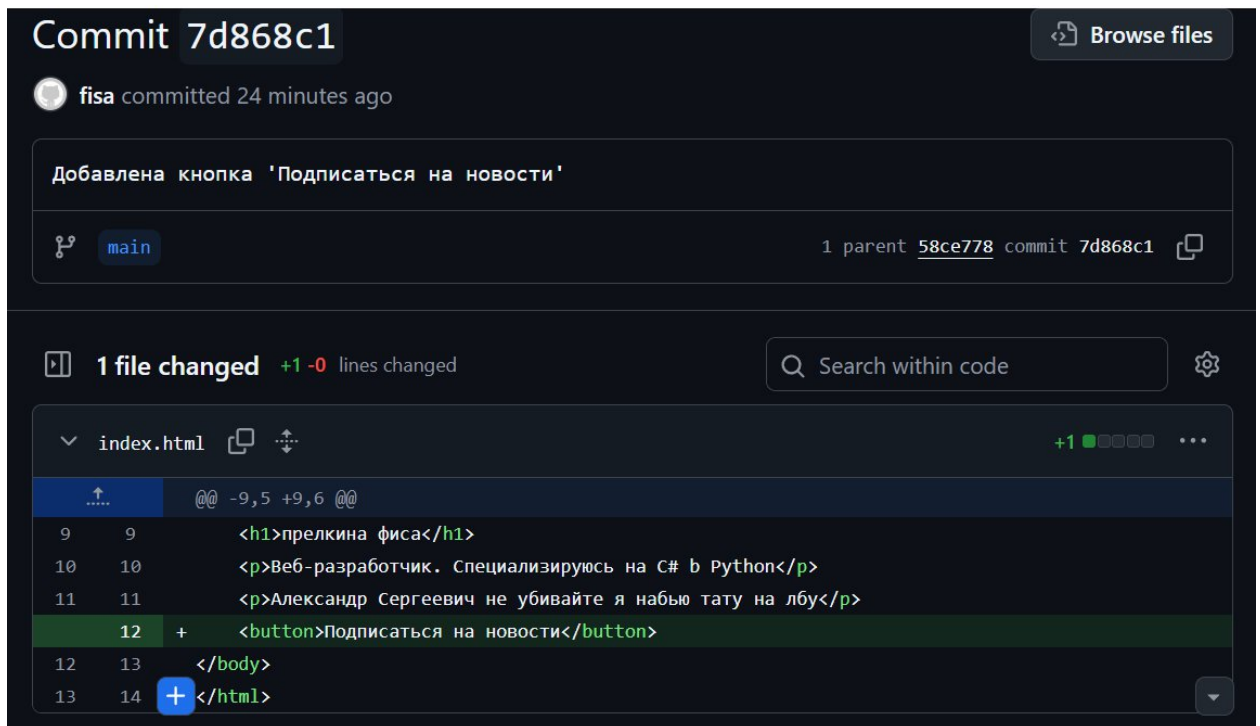


Видим что были добавлены ветки



Вывела второй коммит из гит хаба

Видим что произошло слияние и что есть родительский коммит и итоговый номер коммита, также видно что поменялось



Создаем ветку теста для создания конфликта

```
User@ MINGW64 /c/D/fisaaa (main)
$ git checkout -b conflict-test
Switched to a new branch 'conflict-test'
```

Изменяем имя кнопки на «Подписаться сейчас!»

```
ox;}>Подписаться сейчас!</button>
```

Добавляем под контроль гит пушим

```
User@ MINGW64 /c/D/fisaaa (conflict-test)
$ git add .

User@ MINGW64 /c/D/fisaaa (conflict-test)
$ git commit -m "Изменено название кнопки в conflict-test"
[conflict-test 4d32271] Изменено название кнопки в conflict-test
1 file changed, 1 insertion(+), 1 deletion(-)
```

Переходим в ветку main

```
User@ MINGW64 /c/D/fisaaa (conflict-test)
$
git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

И добавляем изменений в эту же строчку

```
x;}>Подписаться</button>
```

Добавляем изменения под контроль гит и коммитим

Слияем с веткой теста конфликта и видим что возник конфликт

```
User@ MINGW64 /c/D/fisaaa (main)
$ git add .

User@ MINGW64 /c/D/fisaaa (main)
$ git commit -m "Изменено название кнопки в main"
[main 64d51c8] Изменено название кнопки в main
1 file changed, 1 insertion(+), 1 deletion(-)

User@ MINGW64 /c/D/fisaaa (main)
$ git merge conflict-test
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

Заходим в файл индекса и видим то что сконфликтовало

```
color: white;
border: none;
Принять текущее изменение | Принять входящее изменение | Принять оба изменения |
<<<<<< HEAD (текущее изменение)
padding: 10px 20px;}>Подписаться</button>
=====
padding: 10px 20px;}>Подписаться сейчас!</button>
>>>>>> conflict-test (входящее изменение)
<div class="social-login">
<button>Войти через Google</button>
```

Выбираем какое в итоге оставляем изменение

И добавляем это под контроль гит и коммитим

```
User@ MINGW64 /c/D/fisaaa (main|MERGING)
$ git add .

User@ MINGW64 /c/D/fisaaa (main|MERGING)
$ git commit -m "Разрешён конфликт в index.html"
[main a79742d] Разрешён конфликт в index.html
```

Пушим на гитхаб

```
User@ MINGW64 /c/D/fisaaa (main)
$ git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 731 bytes | 365.00 KiB/s, done.
Total 7 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 1 local object.
To https://github.com/fisyarik/fisaaa.git
df4bd83..a79742d  main -> main
```

Заходим на гит хаб и виидм в коммитах все сохраненные коммиты

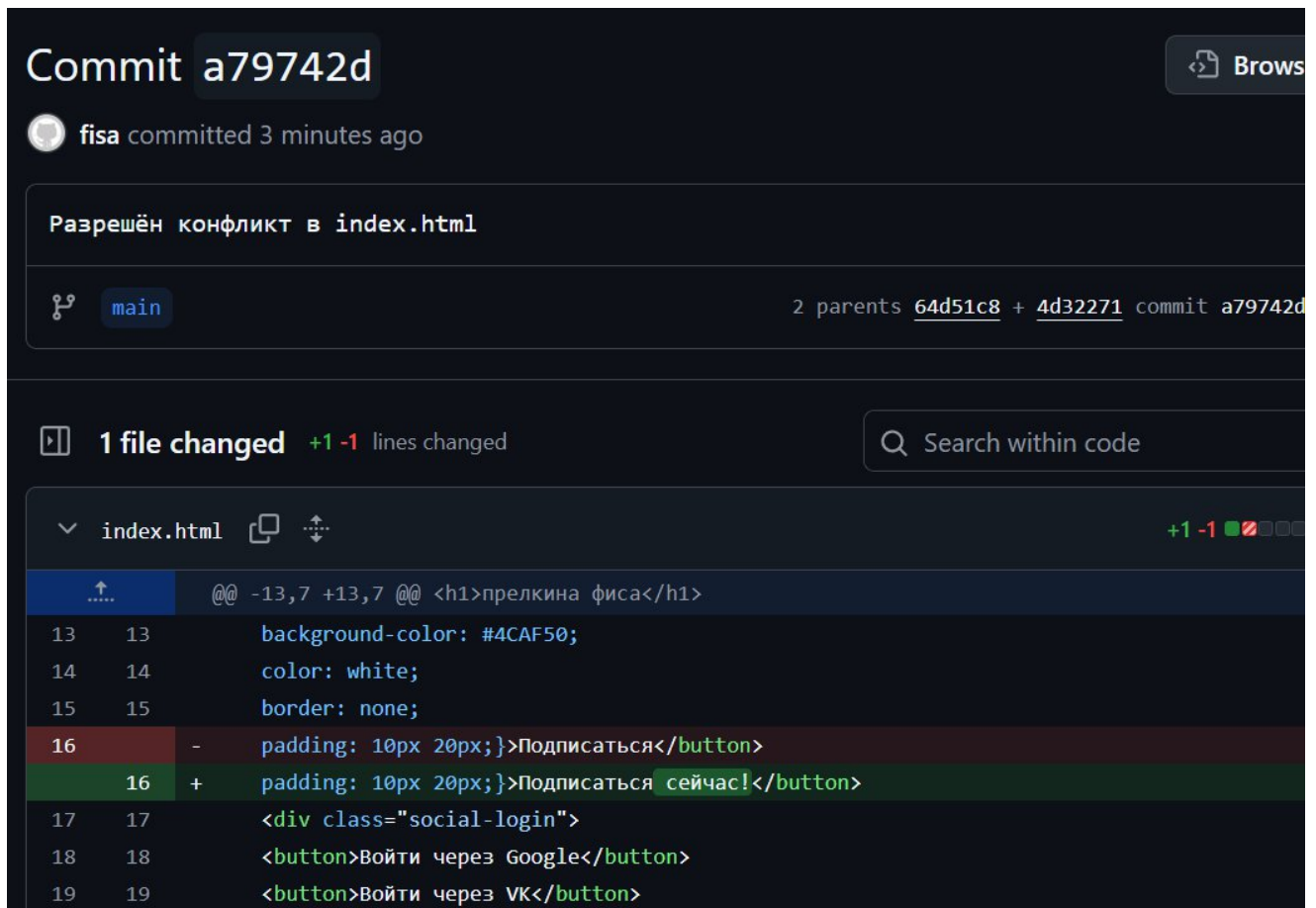
Commits

main ⌵ All users ⌵ All time ⌵

Commits on Feb 8, 2026

Разрешён конфликт в index.html fisa committed 2 minutes ago	a79742d 📄 <>
Изменено название кнопки в main fisa committed 5 minutes ago	64d51c8 📄 <>
Изменено название кнопки в conflict-test fisa committed 9 minutes ago	4d32271 📄 <>

Так же видим коммит с разрешением конфликта



Ссылка на репозиторий гитхаба <https://github.com/fisyarik/fisaaa.git>

Контрольные вопросы:

1. **Какая ветка создаётся сразу после создания репозитория?** Сразу после создания репозитория создаётся ветка `MAIN` (или `MASTER` в старых проектах).
2. **Как задать новую ветку?** Новая ветка задаётся командой `GIT BRANCH <ИМЯ ВЕТКИ>` после чего нужно переключиться на неё с помощью `GIT CHECKOUT <ИМЯ ВЕТКИ>`
3. **Что такое HEAD и какая взаимосвязь между git checkout?** `HEAD` — это указатель на текущий коммит или ветку. Команда `GIT CHECKOUT` меняет указатель `HEAD` на другую ветку или коммит.
4. **Как произвести слияние веток?** Слияние веток выполняется командой `GIT MERGE <ИМЯ ВЕТКИ>`
5. **Что значит fast-forward?** `Fast-forward` — это режим слияния, при котором Git просто перемещает указатель `HEAD` на последний коммит ветки без создания дополнительного коммита слияния (при отсутствии конфликтов и линейной истории).